Instituto Federal do Norte de Minas Gerais - Campus Salinas
Bacharelado em Sistemas de Informação

Sistema Web para Agendamento de Espaços Físicos no IFNMG – Salinas

Aluno: Murilo Santos Gonçalves

Orientador: Arthur Farias Porto

Coorientadora: Jéssica Glória

15 de setembro de 2026

1 INTRODUÇÃO

A organização dos espaços físicos num campus universitário pode aparentar simplicidade, mas revela-se complexa na prática. Quando laboratórios e salas especiais são compartilhados por vários docentes sem um sistema de agendamento adequado, surgem conflitos de horário e a tarefa de controlar quem usa o quê e quando recai sobre servidores que já têm outras atribuições.

No IFNMG – Campus Salinas, esse problema tem contorno específico. O sistema utilizado para agendamentos exige que todos os horários já ocupados sejam cadastrados manualmente antes de qualquer reserva, um processo que precisa ser refeito a cada troca de semestre, uma vez que as atividades dos espaços variam significativamente ao longo do período letivo. Além do retrabalho contínuo, os docentes não têm autonomia para consultar disponibilidade ou solicitar reservas sem intermediação, e a instituição não dispõe de registro consolidado sobre a utilização dos laboratórios ao longo do tempo. O controle permanece, assim, descentralizado e concentrado em um cadastro manual refeito a cada início de semestre. Esse cenário não é exclusivo do Campus Salinas: estudo de Cruz e

Barroso (2023) no IFNMG Campus Almenara identificou a ausência de planejamento permanente e de mecanismos para medir a utilização dos laboratórios.

Este trabalho propõe o desenvolvimento de um sistema web para o gerenciamento de reservas de espaços físicos no IFNMG – Campus Salinas. A proposta visa permitir que docentes consultem a disponibilidade e realizem solicitações de reserva de forma autônoma, enquanto o servidor responsável pelos agendamentos acessa ferramentas de aprovação, recebe notificações por e-mail e gera relatórios sobre o uso dos espaços.

Antes de optar pelo desenvolvimento de uma solução própria, foram investigadas alternativas já disponíveis no mercado e na comunidade de software livre, incluindo plataformas comerciais de reserva de espaços, como o Skedda (Skedda, 2025) e o Microsoft Bookings (Microsoft, 2025), além de sistemas acadêmicos desenvolvidos por outras instituições de ensino (CRUZ; BARROSO, 2023; NASCIMENTO; RIOS; BALUZ, 2020). Essa investigação evidenciou que nenhuma dessas soluções atende ao que a Diretora de Ensino Técnico do campus, responsável pelo projeto, apontou como requisito central deste contexto: fazer com que a disponibilidade dos espaços reflita automaticamente a grade de horários institucional. A comparação detalhada entre essas alternativas e a proposta deste trabalho é apresentada na Seção 2.2.

A necessidade de uma solução específica para esse contexto foi expressa diretamente pela servidora responsável pelo gerenciamento dos agendamentos no campus. Em entrevista concedida durante a fase de desenvolvimento deste trabalho, a Diretora de Ensino Técnico do IFNMG – Campus Salinas, coorientadora deste trabalho, destacou[1]:

> Precisamos de um software que forneça um mapa de utilização dos espaços do campus e permita que horários livres dos espaços sejam agendados pelos docentes, aprovados pelos administradores e visualizados por todos.

A pesquisa é de natureza aplicada, pois resulta em um produto para um problema identificado num contexto real. É descritiva quanto aos objetivos, e os procedimentos incluem pesquisa bibliográfica e estudo de caso. Espera-se que o sistema contribua para a otimização do uso dos espaços físicos, a redução de conflitos de agendamento e a melhoria na gestão institucional do campus.

1.1 OBJETIVO GERAL

Desenvolver um sistema web para o gerenciamento de reservas de espaços físicos no IFNMG – Campus Salinas.

[1] Informação verbal fornecida pela Diretora de Ensino Técnico do IFNMG – Campus Salinas, em entrevistas realizadas em julho de 2025, novembro de 2025 e abril de 2026.

1.2 OBJETIVOS ESPECÍFICOS

- Levantar o processo atual de agendamento e identificar as necessidades dos perfis de usuários envolvidos;
- Especificar os requisitos funcionais e não funcionais do sistema;
- Planejar a implementação do sistema web, definindo as tecnologias e a arquitetura técnica a serem adotadas, com base nos requisitos especificados;
- Disponibilizar ao administrador ferramentas para a gestão dos agendamentos e o acompanhamento do uso dos espaços.

1.3 ESTRUTURA DO TRABALHO

Este trabalho está estruturado da seguinte forma: no Capítulo 2 é apresentada a revisão bibliográfica, que abrange o contexto institucional, os trabalhos relacionados e a fundamentação tecnológica; no Capítulo 3, a metodologia utilizada na condução da pesquisa; no Capítulo 4, o cronograma de execução das etapas previstas; e, por fim, as referências bibliográficas consultadas.

2 REVISÃO BIBLIOGRÁFICA

Este capítulo apresenta o referencial teórico que embasa o problema e a solução propostos. A primeira parte situa o leitor no contexto institucional em que o problema ocorre: o IFNMG – Campus Salinas e a forma como instituições de ensino lidam com a gestão de espaços físicos. O processo de agendamento atualmente em uso no campus, por ser resultado do levantamento realizado nesta pesquisa, é caracterizado no capítulo de metodologia. A segunda parte apresenta os trabalhos relacionados, descrevendo iniciativas semelhantes já documentadas na literatura e identificando em que pontos o presente trabalho se distingue delas. A terceira parte reúne os conceitos técnicos que fundamentam o desenvolvimento do sistema web proposto, cobrindo engenharia de software, modelagem com UML, banco de dados, controle de acesso, arquitetura web e usabilidade.

2.1 CONTEXTO INSTITUCIONAL

Esta seção descreve o IFNMG – Campus Salinas e o funcionamento dos seus espaços físicos e apresenta o que a literatura registra sobre gestão de espaços em ambientes acadêmicos.

2.1.1 O IFNMG E O CAMPUS SALINAS

O Instituto Federal de Educação, Ciência e Tecnologia do Norte de Minas Gerais (IFNMG) é uma autarquia federal criada pela Lei n.º 11.892, de 29 de dezembro de 2008, que reorganizou a Rede Federal de Educação Profissional, Científica e Tecnológica no Brasil. Conforme seu Plano de Desenvolvimento Institucional, o IFNMG tem por missão promover educação profissional, científica e tecnológica por meio do ensino, da pesquisa e da extensão, com vistas à formação humana integral e ao desenvolvimento regional (Instituto Federal do Norte de Minas Gerais, 2024).

O Campus Salinas integra a rede do IFNMG e oferta cursos em diferentes níveis e modalidades, entre eles o Bacharelado em Sistemas de Informação, no âmbito do qual este trabalho está inserido. Para o funcionamento dessas atividades acadêmicas, o campus dispõe de laboratórios de informática e outras salas especiais cujo uso é compartilhado por diferentes turmas e docentes ao longo do período letivo, o que exige mecanismos de organização e controle sobre a ocupação desses espaços.

A natureza dos institutos federais agrava esse desafio. Ao contrário de instituições que ofertam um único tipo de curso, os IFs atendem simultaneamente a turmas de educação técnica integrada ao ensino médio, cursos técnicos subsequentes e cursos de nível superior, todos com horários distintos e, em muitos casos, disputando os mesmos laboratórios. Um laboratório de informática, por exemplo, pode ser utilizado em horários diferentes para aulas de disciplinas técnicas, para disciplinas do bacharelado e para atividades de extensão, o que torna a gestão manual da disponibilidade um trabalho de complexidade operacional (Instituto Federal do Norte de Minas Gerais, 2024).

2.1.2 A GESTÃO DE ESPAÇOS EM INSTITUIÇÕES DE ENSINO

A gestão dos espaços físicos em instituições de ensino inclui o planejamento do uso, o controle de disponibilidade, o registro de ocupações e o acompanhamento de como os recursos são efetivamente utilizados. Quando esses mecanismos não existem ou são precários, o resultado é a subutilização dos espaços e a dificuldade de responder a demandas concretas dos usuários.

Cruz e Barroso (2023), em pesquisa realizada no IFNMG Campus Almenara, investigaram a gestão de laboratórios educacionais e identificaram que “não existe um planejamento oficial permanente nesses espaços, a gestão do uso ocorre apenas através da reserva de espaço com antecedência”. Os autores também constataram que não foram identificados métodos de gestão do desempenho dos laboratórios nem formas de medir a utilização eficiente desses espaços, o que evidencia que a ausência de ferramentas adequadas de gestão não é uma particularidade de um único campus.

Problema semelhante foi documentado por Nascimento, Rios e Baluz (2020) na

Universidade Estadual do Piauí, Campus Parnaíba, onde as reservas de recursos físicos eram registradas manualmente pelos servidores por meio de uma agenda digital. A demanda tornava o processo oneroso para a equipe administrativa, o que motivou o desenvolvimento de um sistema web para controlar as solicitações. Os resultados do trabalho apontaram satisfação positiva dos usuários com a solução desenvolvida. Esse resultado demonstra que sistemas digitais de reserva são viáveis e bem recebidos em ambientes acadêmicos.

O que os dois trabalhos revelam, em conjunto, é que a ausência de sistemas adequados para gestão de espaços em instituições de ensino não é um problema isolado. Trata-se de uma lacuna recorrente em campi públicos brasileiros, onde o processo manual se sustenta pela força de hábito e pela ausência de alternativas, mas produz retrabalho, limita a autonomia dos usuários e não gera dados que possam subsidiar o planejamento institucional.

2.2 TRABALHOS RELACIONADOS

Esta seção apresenta trabalhos e soluções que trataram de problemas semelhantes ao abordado nesta pesquisa, seja pelo diagnóstico da gestão de espaços em instituições federais de ensino, seja pelo desenvolvimento de sistemas de reserva em ambientes acadêmicos, seja pela oferta de ferramentas de mercado para a reserva de espaços. A análise desses trabalhos e soluções permite situar a presente proposta em relação ao que já foi produzido e identificar os aspectos que a diferenciam.

2.2.1 SOLUÇÕES EXISTENTES DE RESERVA DE ESPAÇOS

Além dos trabalhos acadêmicos, foram analisadas soluções de software já disponíveis para a reserva de espaços, com o objetivo de verificar se alguma delas resolveria o problema do Campus Salinas sem a necessidade de desenvolvimento de um sistema específico. A análise concentrou-se em duas plataformas comerciais representativas: o Skedda e o Microsoft Bookings.

O Skedda é uma plataforma web dedicada à reserva de espaços físicos, como salas, escritórios e laboratórios, que oferece autoatendimento aos usuários, definição de regras de reserva e visualização de disponibilidade em formato de agenda (Skedda, 2025). O Microsoft Bookings, integrante do pacote Microsoft 365, é voltado ao agendamento de compromissos e ao gerenciamento de calendários, com reserva de recursos por meio de caixas de correio associadas a salas (Microsoft, 2025). Ambas as ferramentas cobrem funcionalidades que também são esperadas do sistema proposto, como a consulta de disponibilidade, a solicitação de reserva pelo próprio usuário e a notificação automática das partes envolvidas, o que demonstra a maturidade desse tipo de aplicação no mercado.

Nenhuma das duas, contudo, resolve o problema central identificado no campus.

O gargalo do processo atual não está na reserva em si, mas na necessidade de que a disponibilidade dos espaços reflita a grade de horários institucional, na qual as aulas regulares já ocupam parte expressiva dos ambientes ao longo do semestre. Como essas plataformas não importam as planilhas de horários produzidas pelo setor responsável, a ocupação decorrente das aulas teria de ser cadastrada manualmente, espaço por espaço, a cada início de semestre, exatamente o trabalho repetitivo que motiva esta proposta. Somese a isso o fato de serem soluções proprietárias, em geral pagas por assinatura e hospedadas em servidores externos, o que adiciona custos recorrentes e reduz o controle da instituição sobre os próprios dados. É justamente a integração com o instrumento de planejamento já utilizado pelo campus, não contemplada por nenhuma das duas plataformas, que torna o desenvolvimento de um sistema próprio mais adequado do que a adoção de uma ferramenta de mercado.

2.2.2 GESTÃO DE LABORATÓRIOS NO IFNMG CAMPUS ALMENARA

Cruz e Barroso (2023) conduziram uma pesquisa de campo no IFNMG Campus Almenara com o objetivo de analisar a utilização dos espaços e equipamentos dos laboratórios educacionais daquela unidade. O estudo adotou abordagem qualitativa e quantitativa, com coleta de dados por meio de questionários aplicados a servidores e docentes do campus. Os resultados revelaram que a gestão dos laboratórios ocorre de forma informal, baseada exclusivamente em reservas pontuais feitas com antecedência, sem planejamento oficial permanente e sem qualquer mecanismo para medir o desempenho ou a eficiência no uso desses espaços.

O trabalho de Cruz e Barroso (2023) contribui para a presente pesquisa ao documentar, em um campus do próprio IFNMG, a mesma ausência de ferramentas de gestão identificada no Campus Salinas. A semelhança entre os dois contextos reforça que o problema não é localizado e que a carência de soluções tecnológicas adequadas é uma realidade institucional mais ampla. No entanto, o trabalho desses autores se limita ao diagnóstico: não propõe nem desenvolve qualquer solução de software para o problema identificado. O presente trabalho avança nessa direção ao propor um sistema que automatiza o controle de disponibilidade e registra o histórico de uso dos espaços.

2.2.3 SISTEMA WEB DE RESERVAS NA UESPI – PARNAÍBA

Nascimento, Rios e Baluz (2020) desenvolveram um sistema web para controlar pedidos de reservas de recursos físicos na Universidade Estadual do Piauí, Campus Parnaíba. O contexto era semelhante ao do presente trabalho: as reservas de espaços e equipamentos eram registradas manualmente por servidores em uma agenda digital, e a demanda tornava o processo oneroso e sujeito a falhas. A solução desenvolvida pelos autores automatizou o controle das solicitações, disponibilizou a interface para os usuários da instituição e obteve

avaliação positiva quanto à satisfação dos usuários com o sistema.

O trabalho de Nascimento, Rios e Baluz (2020) demonstra que o desenvolvimento de sistemas web de reserva em instituições de ensino é viável e resulta em ganhos concretos para a gestão. A presente proposta compartilha o mesmo problema central, mas dele se distingue pela importação das planilhas de horários institucionais, pelos perfis de acesso baseados no modelo RBAC, pelas notificações automáticas por e-mail no fluxo de aprovação e pela geração de relatórios de uso dos espaços. A Tabela 1 sintetiza essa comparação, estendida também às duas plataformas comerciais analisadas na Seção 2.2.1.

Tabela 1 – Comparação entre as soluções analisadas e o sistema proposto

Funcionalidade	**Skedda**	**Bookings**	**Cruz et al.**	**Nascimento et al.**	**Proposto**
Consulta de disponibilidade pelo usuário	Sim	Sim	N/A	Sim	Sim
Solicitação de reserva sem intermediação	Sim	Sim	N/A	Sim	Sim
Aprovação ou rejeição da solicitação	Sim	Sim	N/A	Sim	Sim
Notificação automática por e-mail	Sim	Sim	N/A	N/I	Sim
Importação da planilha de horários institucional	Não	Não	N/A	Não	Sim
Atualização da ocupação ao longo do semestre	Não	Não	N/A	Não	Sim
Relatórios de uso dos espaços	N/I	N/I	Não	N/I	Sim
Controle de acesso por papéis (RBAC)	N/I	N/I	N/A	N/I	Sim
Consulta pública sem autenticação	Não	Não	N/A	N/I	Sim
Sem custo de licença ou assinatura	Não	Não	N/A	Sim	Sim
Hospedagem sob controle da instituição	Não	Não	N/A	Sim	Sim

N/A: não se aplica, por se tratar de estudo diagnóstico sem desenvolvimento de sistema. N/I: não informado pela fonte consultada. Fonte: Elaborado pelo autor (2026).

2.3 FUNDAMENTAÇÃO TECNOLÓGICA

Esta seção apresenta os conceitos técnicos que embasam o desenvolvimento do sistema proposto, organizados da especificação e modelagem do software até os aspectos de interface com o usuário, com passagem por banco de dados, controle de acesso e arquitetura web.

2.3.1 ENGENHARIA DE SOFTWARE

Sommerville (2018) define software como programas de computador e documentação associada, que pode ser desenvolvido para um cliente específico ou para o mercado em geral. A engenharia de software é a disciplina que se ocupa de todos os aspectos da produção de software, desde a especificação inicial do sistema até as necessidades de manutenção após a sua implantação. Deste campo, este trabalho se apoia diretamente em dois pontos: a especificação de requisitos e os atributos de qualidade que orientam as decisões de projeto.

Um conceito central dessa disciplina é a especificação de requisitos. Sommerville (2018) distingue os requisitos funcionais dos não funcionais: os primeiros descrevem as funções que o sistema deve executar e como deve se comportar em situações específicas; os segundos estabelecem restrições ao sistema, como desempenho, segurança e disponibilidade.

Sommerville (2018) também descreve atributos que um sistema de software profissional precisa apresentar para ser considerado de boa qualidade: manutenibilidade, ou seja, a capacidade de evoluir para atender a necessidades futuras; confiabilidade e segurança, que inclui comportamento correto e proteção contra falhas e acessos indevidos; eficiência, que diz respeito ao uso criterioso dos recursos de hardware; e aceitabilidade, que significa ser compreensível e compatível com o que os usuários esperam.

2.3.2 MODELAGEM DE SISTEMAS COM UML

A *Unified Modeling Language* (UML) é uma linguagem de modelagem padronizada para especificar, visualizar e documentar sistemas de software orientados a objetos (BEZERRA, 2007). Bezerra (2007) esclarece que a UML não é uma metodologia de desenvolvimento, mas uma notação que pode ser adotada em conjunto com diferentes processos. Ela oferece um conjunto de diagramas que representam aspectos distintos do sistema, como a sua estrutura estática e o seu comportamento dinâmico.

Entre os diagramas oferecidos pela UML, este trabalho emprega o diagrama de casos de uso, que descreve as funcionalidades do sistema do ponto de vista de quem interage com ele: representa os atores (usuários ou sistemas externos), os casos de uso (as funções que o sistema oferece) e os relacionamentos entre eles, delimitando o escopo funcional do sistema sem detalhar como cada função é implementada (LARMAN, 2007). Os diagramas de classes e de sequência, que representam respectivamente a estrutura estática e o comportamento dinâmico do sistema, serão elaborados no TCC 2, na fase de implementação.

Larman (2007) destaca que a modelagem com UML permite registrar e comunicar decisões de projeto de forma visual antes de qualquer implementação, o que facilita o alinhamento entre desenvolvedores e demais partes envolvidas.

2.3.3 BANCO DE DADOS RELACIONAL

Um banco de dados é uma coleção organizada de dados inter-relacionados, gerenciada por um sistema que permite armazená-los, recuperá-los e manipulá-los com eficiência (HEUSER, 2009). Heuser (2009) apresenta o modelo entidade-relacionamento como a principal ferramenta para o projeto conceitual de bancos de dados, por permitir representar as entidades do domínio e os vínculos entre elas antes de definir como os dados serão fisicamente armazenados.

O modelo relacional organiza os dados em tabelas formadas por linhas e colunas, onde cada tabela representa uma entidade e cada linha, um registro dessa entidade. A integridade dos dados é assegurada por restrições como chaves primárias e estrangeiras, que garantem a unicidade dos registros e os vínculos entre tabelas (SILBERSCHATZ; KORTH; SUDARSHAN, 2012). Heuser (2009) também descreve o processo de normalização,

pelo qual o esquema relacional é reorganizado para eliminar redundâncias e anomalias de atualização. Um banco de dados normalizado garante que cada informação seja armazenada em um único lugar, reduzindo o risco de inconsistências quando os dados precisam ser modificados.

Silberschatz, Korth e Sudarshan (2012) definem um sistema gerenciador de banco de dados (SGBD) como um conjunto de programas que permite criar, manter e acessar um banco de dados. Os autores também descrevem a linguagem SQL (*Structured Query Language*) como o padrão para definir esquemas, inserir dados e realizar consultas em bancos de dados relacionais. A SQL combina uma linguagem de definição de dados, para criar e alterar tabelas e restrições, com uma linguagem de manipulação de dados, para inserir, atualizar, remover e consultar registros. Para o sistema proposto, o uso de um SGBD relacional é adequado porque os dados a serem gerenciados (espaços físicos, horários, usuários, solicitações de reserva) têm estrutura bem definida e mantêm entre si relacionamentos que o modelo relacional representa com precisão.

Cabe distinguir o papel da planilha de horários do papel do banco de dados nessa arquitetura. A planilha institucional permanece como a fonte da ocupação decorrente das aulas regulares e é lida pelo sistema no momento da importação, mas não substitui o banco. É no banco que ficam o histórico de utilização que sustenta os relatórios, os estados pelos quais passa cada solicitação no fluxo de aprovação, o vínculo entre usuário, papel e reserva exigido pelo controle de acesso e o tratamento de solicitações simultâneas, que em um arquivo de planilha estaria sujeito a sobrescrita.

Entre os SGBDs relacionais disponíveis, o PostgreSQL é um sistema de código aberto com mais de três décadas de desenvolvimento, reconhecido pela conformidade com o padrão SQL e pela robustez no tratamento de transações. Ele garante as propriedades ACID (atomicidade, consistência, isolamento e durabilidade), oferece suporte a tipos de dados avançados, controle de concorrência multiversão (MVCC) e recursos como busca textual e integridade referencial, características que o tornam adequado a aplicações que exigem consistência e confiabilidade no armazenamento dos dados (The PostgreSQL Global Development Group, 2024).

2.3.4 CONTROLE DE ACESSO BASEADO EM PAPÉIS

O controle de acesso define quais usuários têm permissão para executar quais operações em um sistema. Entre os modelos existentes, o *Role-Based Access Control* (RBAC) proposto por Sandhu et al. (1996) é amplamente adotado em sistemas de informação institucionais.

No modelo RBAC, os direitos de acesso são associados a papéis, e os usuários adquirem esses direitos ao serem atribuídos a um ou mais papéis. Sandhu et al. (1996) descrevem que essa abordagem simplifica a administração, pois as permissões são definidas

no nível do papel e não para cada usuário individualmente. Quando as responsabilidades de um usuário mudam, basta alterar a sua atribuição de papéis, sem necessidade de revisar cada permissão específica.

Sandhu et al. (1996) definem quatro modelos de referência que compõem a família RBAC. O RBAC0, o modelo base, define os elementos fundamentais: usuários, papéis, permissões e sessões, onde uma sessão corresponde a um período de atividade de um usuário dentro de um subconjunto dos seus papéis. O RBAC1 estende o modelo base com hierarquias de papéis, permitindo que papéis mais abrangentes herdem as permissões de papéis subordinados. O RBAC2 acrescenta restrições ao RBAC0, incluindo o conceito de separação de obrigações, pelo qual um usuário não pode acumular simultaneamente papéis conflitantes. O RBAC3 combina hierarquias e restrições, reunindo as características dos modelos anteriores. Para os fins deste trabalho, o modelo base (RBAC0) com dois papéis autenticados sem hierarquia entre eles, somado a uma rota pública de consulta, é suficiente para atender aos requisitos de controle de acesso identificados.

O sistema proposto neste trabalho adota esse modelo com dois papéis autenticados: Administrador, responsável pela gestão geral do sistema e dos usuários; e Docente, que pode consultar disponibilidade e solicitar reservas. A consulta pública de disponibilidade é acessível sem autenticação, por meio de uma rota aberta, exibindo apenas os horários livres e ocupados sem expor dados pessoais de docentes ou detalhes de reservas individuais. Essa definição orienta diretamente a especificação dos requisitos funcionais e a modelagem do sistema.

2.3.5 DESENVOLVIMENTO WEB E ARQUITETURA DE SOFTWARE

Um sistema web é uma aplicação de software acessada por meio de um navegador, que utiliza a infraestrutura da internet para a comunicação entre clientes e servidores. Tanenbaum e Wetherall (2011) descrevem a *World Wide Web* como um sistema de documentos interconectados distribuídos pela internet, baseado no protocolo HTTP para a troca de informações. Esse modelo de comunicação é a base sobre a qual sistemas web são construídos.

Do ponto de vista arquitetural, Laudon e Laudon (2014) descrevem que sistemas de informação baseados na web se organizam tipicamente em três camadas: a de apresentação, responsável pela interface com o usuário no navegador; a de aplicação, onde ficam as regras de negócio; e a de dados, onde as informações são armazenadas. Essa separação em camadas facilita a manutenção e permite que cada parte do sistema seja modificada sem afetar diretamente as demais.

Tanenbaum e Wetherall (2011) descrevem o protocolo HTTP como uma especificação sem estado (*stateless*), o que significa que cada requisição do cliente ao servidor é tratada de forma independente, sem que o servidor retenha informações sobre interações

anteriores. Essa característica simplifica a arquitetura dos servidores web, mas exige que o estado da aplicação, como a sessão do usuário autenticado, seja gerenciado explicitamente pela camada de aplicação, por meio de mecanismos como *tokens* de autenticação.

Laudon e Laudon (2014) também apontam que sistemas web têm a vantagem de não exigir instalação de software no dispositivo do usuário: basta um navegador com acesso à internet para que o sistema esteja disponível. Isso reduz os custos de implantação, elimina problemas de compatibilidade entre sistemas operacionais e centraliza a manutenção no servidor, onde atualizações passam a valer imediatamente para todos os usuários. No caso do sistema proposto, essa característica é relevante porque os docentes do campus utilizam dispositivos variados e não dependem de nenhuma configuração específica para acessar a plataforma de reservas.

Para o sistema proposto, a arquitetura em três camadas é adequada porque isola a interface utilizada pelos docentes da lógica que controla as reservas e da base de dados onde os registros são mantidos. Na prática, essa separação é implementada por meio de um *framework* web, que fornece a estrutura de código e os mecanismos de comunicação entre as camadas, reduzindo o trabalho de infraestrutura e permitindo que o desenvolvimento se concentre nas regras de negócio da aplicação. O padrão arquitetural adotado pelo *framework* escolhido para este sistema é detalhado na seção a seguir.

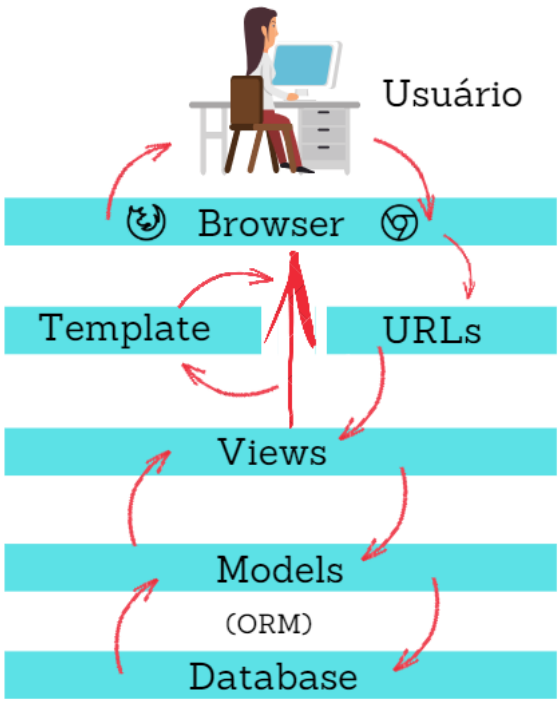
2.3.6 PADRÕES ARQUITETURAIS PARA APLICAÇÕES WEB: MVC E MTV

O padrão *Model-View-Controller* (MVC) é uma abordagem arquitetural consolidada para organizar o código de aplicações interativas em três componentes com responsabilidades bem delimitadas. O *Model* gerencia os dados e as regras de negócio da aplicação. A *View* é responsável pela apresentação das informações ao usuário. O *Controller* recebe as ações do usuário, processa as entradas e coordena a interação entre *Model* e *View* (Django Software Foundation, 2024). Essa separação permite que cada componente seja desenvolvido e testado de forma independente.

O *framework* Django adota uma variação desse padrão denominada MTV (*Model-Template-View*), que redistribui as responsabilidades de forma distinta. O *Model* cumpre a mesma função do MVC: representa as entidades de dados e encapsula as regras de negócio associadas ao banco de dados. O *Template* substitui a *View* do MVC e se ocupa exclusivamente da renderização da interface, gerando o HTML entregue ao navegador. A *View* do MTV, por sua vez, assume o papel que no MVC pertenceria ao *Controller*: ela recebe a requisição HTTP, aciona os *Models* necessários e seleciona o *Template* adequado para compor a resposta (TreinaWeb, 2023). A Figura 1 representa essa organização.

A diferença mais relevante entre os dois padrões está na forma como o roteamento de requisições é gerenciado. No MVC clássico, o *Controller* é o ponto de entrada das requisições do cliente. No MTV do Django, esse papel é desempenhado pelo próprio

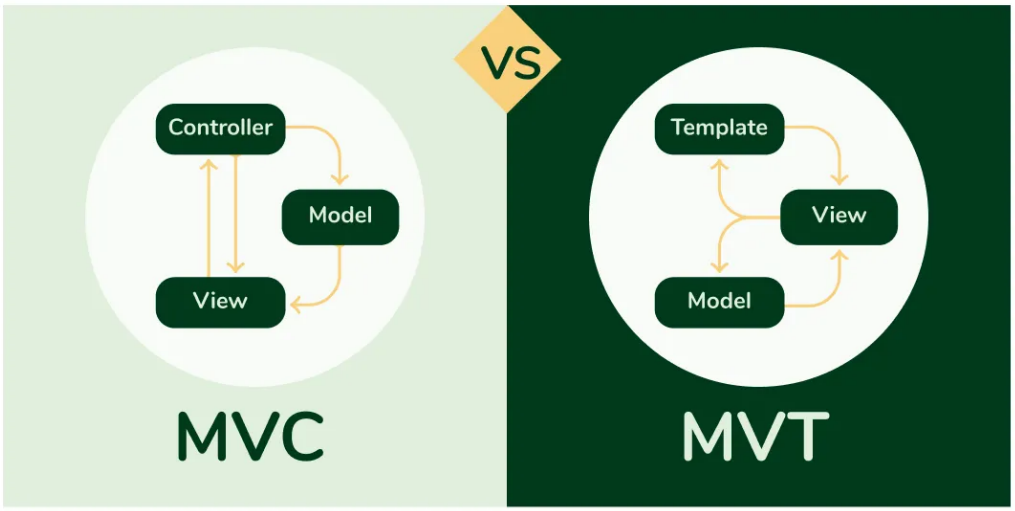
Figura 1 – Arquitetura MTV do Django



Fonte: (TreinaWeb, 2023)

mecanismo de roteamento de URLs do *framework*, que mapeia cada endereço recebido para a *View* correspondente por meio do arquivo `urls.py`, antes mesmo de qualquer lógica da aplicação ser acionada (Django Software Foundation, 2024). A Figura 2 contrasta a distribuição de responsabilidades nos dois padrões.

Figura 2 – Comparação entre as arquiteturas MVC e MTV

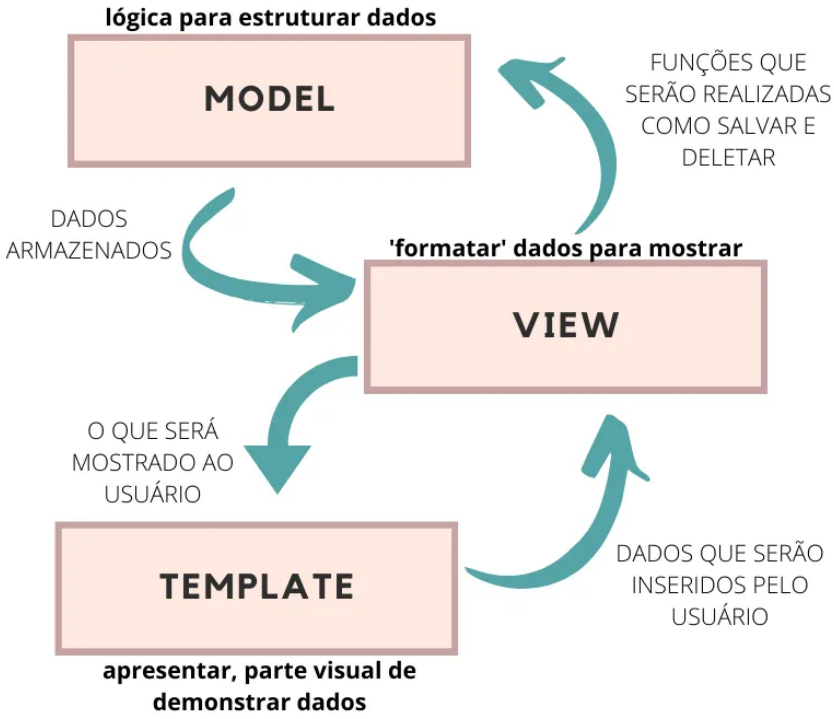


Fonte: (Django Software Foundation, 2024)

O fluxo de uma requisição no Django percorre essas camadas de forma sequencial: o cliente envia uma requisição HTTP, o roteador de URLs identifica a *View* responsável, a

View acessa os *Models* para obter ou persistir dados e, em seguida, seleciona o *Template* adequado para gerar a resposta enviada de volta ao cliente (SILVA, 2023). Esse percurso é ilustrado na Figura 3.

Figura 3 – Fluxo de requisição na arquitetura MTV do Django



Fonte: (SILVA, 2023)

Essa organização em camadas sustenta a escolha do Django como *framework* para o sistema proposto. A separação clara entre dados (*Model*), lógica de controle (*View*) e apresentação (*Template*) permite que cada parte do sistema seja mantida e evoluída de forma independente, o que é especialmente relevante em um projeto de TCC cujo desenvolvimento continuará no semestre seguinte.

2.3.7 O FRAMEWORK DJANGO

O Django é um *framework* web de código aberto escrito em Python. Ele opera segundo o princípio *batteries included*: disponibiliza nativamente os componentes mais comuns no desenvolvimento web, como ORM, sistema de autenticação, roteamento de URLs, mecanismo de *templates* e envio de e-mails, sem dependência de bibliotecas externas para essas funções (Django Software Foundation, 2024). Outro princípio central do *framework* é o *Don't Repeat Yourself* (DRY), que orienta o reaproveitamento de código e a centralização de lógicas que, se duplicadas, aumentariam o custo de manutenção ao longo do tempo (Django Software Foundation, 2024).

O roteamento de URLs é configurado no arquivo `urls.py`, que associa cada padrão de endereço a uma *view* específica por meio da função `path()`. Quando uma requisição

chega ao servidor, o *framework* percorre a lista de padrões registrados até encontrar o primeiro que corresponda à URL solicitada e delega o processamento à *view* associada. Esse mecanismo é independente da estrutura de diretórios da aplicação, o que permite organizar as URLs de forma legível e separada da lógica de negócio (Django Software Foundation, 2024).

Cada *model* do Django é uma classe Python que herda de `django.db.models.Model`. Cada atributo da classe representa uma coluna da tabela correspondente no banco de dados, e o tipo do campo, como `CharField`, `DateTimeField` ou `BooleanField`, determina o tipo de dado armazenado e as validações aplicadas (Django Software Foundation, 2024). Relacionamentos entre entidades são representados por campos específicos: `ForeignKey` para associações muitos-para-um e `ManyToManyField` para associações muitos-para-muitos. O ORM traduz as operações sobre esses objetos Python em comandos SQL compatíveis com o banco de dados configurado (Django Software Foundation, 2024).

As mudanças nos *models* são controladas pelo sistema de *migrations*. O comando `makemigrations` gera um arquivo descrevendo as alterações a serem aplicadas no banco; o comando `migrate` as aplica de forma incremental e rastreável, mantendo um histórico completo de todas as versões do esquema (Django Software Foundation, 2024).

O sistema de *templates* do Django utiliza a *Django Template Language* (DTL), que insere dados dinâmicos nas páginas HTML por meio de variáveis e *tags* de controle. A herança de *templates*, obtida com as *tags* `{% extends %}` e `{% block %}`, permite que um *template* base defina a estrutura comum a todas as páginas, como cabeçalho, menu e rodapé, e que os demais *templates* sobrescrevam apenas os blocos de conteúdo específicos (Django Software Foundation, 2024). Esse recurso elimina duplicação de HTML e garante consistência visual em toda a aplicação.

O Django oferece proteções de segurança ativas por padrão. A proteção contra *Cross-Site Request Forgery* (CSRF) é implementada por um *middleware* que exige um *token* válido em todas as requisições que modificam dados no servidor; a *tag* `{% csrf_token %}` insere esse *token* automaticamente nos formulários. A proteção contra *Cross-Site Scripting* (XSS) é garantida pelo sistema de *templates*, que escapa automaticamente todo conteúdo dinâmico antes de inseri-lo no HTML. A proteção contra injeção de SQL é inerente ao ORM, que parametriza internamente as consultas geradas, de modo que dados fornecidos pelo usuário nunca são interpolados diretamente no SQL (Django Software Foundation, 2024).

2.3.8 USABILIDADE E INTERFACE DO USUÁRIO

Usabilidade é a medida em que um produto pode ser usado por pessoas específicas para atingir objetivos específicos com eficácia, eficiência e satisfação num determinado contexto (NIELSEN, 1993). Nielsen (1993) descreve cinco atributos que compõem a

usabilidade de um sistema: facilidade de aprendizado, eficiência de uso, facilidade de memorização, baixa taxa de erros e satisfação subjetiva do usuário. Um sistema com boa usabilidade permite que o usuário realize suas tarefas sem depender de treinamento extensivo nem de suporte constante.

Para orientar o projeto de interfaces, Nielsen (1994) propôs dez heurísticas de usabilidade que se tornaram referência na área: (1) visibilidade do estado do sistema, mantendo o usuário informado sobre o que está acontecendo; (2) correspondência entre o sistema e o mundo real, usando linguagem e convenções familiares ao usuário; (3) controle e liberdade do usuário, permitindo desfazer ações e sair de estados indesejados; (4) consistência e padrões, seguindo convenções da plataforma para que o usuário não precise aprender novos significados; (5) prevenção de erros, projetando para evitar que erros ocorram; (6) reconhecimento em vez de memorização, tornando objetos e opções visíveis para reduzir a carga de memória do usuário; (7) flexibilidade e eficiência de uso, permitindo que usuários experientes realizem tarefas mais rapidamente; (8) estética e design minimalista, evitando informações irrelevantes que competem com o conteúdo principal; (9) ajuda para reconhecer, diagnosticar e recuperar erros, exibindo mensagens claras sobre o que deu errado e como corrigir; e (10) ajuda e documentação, oferecendo suporte quando necessário, de forma fácil de buscar e orientada à tarefa.

Algumas dessas heurísticas têm peso particular no sistema proposto. A visibilidade do estado do sistema é importante porque o docente precisa saber, a qualquer momento, se a sua solicitação de reserva está pendente, aprovada ou rejeitada. A correspondência com o mundo real orienta o uso de termos como “laboratório”, “reserva” e “disponível” em vez de jargões técnicos. A prevenção de erros, por sua vez, inclui mecanismos como a verificação automática de conflitos de horário antes de permitir a solicitação.

No contexto deste trabalho, usabilidade é relevante porque o sistema será utilizado por docentes com familiaridade tecnológica variada. Um sistema que exija treinamento extensivo ou que gere dúvidas frequentes tende a ser abandonado, o que comprometeria o próprio objetivo de substituir o processo manual de agendamento.

2.3.9 TESTES DE SOFTWARE

Testes de software são atividades planejadas com o objetivo de verificar se um sistema se comporta conforme o esperado e identificar defeitos antes que o produto chegue ao usuário final. Sommerville (2018) define teste como um processo de validação e verificação que busca demonstrar que o software atende aos seus requisitos e identificar situações em que ele se comporta de forma incorreta ou inesperada. Pressman e Maxim (2016) complementam essa perspectiva ao afirmar que testar é executar um programa com a intenção explícita de encontrar erros, distinguindo o processo de testes de outras atividades de garantia de qualidade por seu caráter sistematicamente verificativo: um teste

bem-sucedido é aquele que revela um defeito até então desconhecido.

Os testes unitários são a forma mais elementar de teste de software e têm como foco verificar o comportamento de unidades individuais de código de forma isolada, antes de integrá-las ao restante do sistema. Myers, Sandler e Badgett (2011) definem teste unitário como o processo de testar subprogramas, funções ou métodos de maneira independente, de modo que, quando uma unidade falha, a localização do defeito seja imediata, sem a necessidade de investigar interações entre múltiplos componentes. Pressman e Maxim (2016) acrescentam que os testes unitários devem cobrir as estruturas de controle internas de cada componente, as condições de contorno, os caminhos independentes de execução e o tratamento de erros.

No contexto de sistemas web com regras de negócio definidas, os testes unitários têm papel direto na validação da lógica da aplicação. Regras como a verificação de conflito de horário antes de confirmar uma reserva, a validação de permissões por perfil de usuário ou o processamento de dados importados de planilhas institucionais são comportamentos que podem ser verificados de forma isolada, independentemente da interface ou do banco de dados. Sommerville (2018) observa que os testes unitários são especialmente eficazes para validar componentes que implementam regras de negócio, pois permitem verificar cada condição individualmente e com precisão. À medida que as unidades validadas são combinadas, os testes de integração verificam se os componentes interagem corretamente entre si (PRESSMAN; MAXIM, 2016).

3 METODOLOGIA

Esta seção descreve a metodologia da pesquisa, organizada em seis partes, todas desenvolvidas no TCC 1. A Seção 3.1 apresenta o tipo de pesquisa quanto à natureza, aos objetivos e ao procedimento técnico. Logo após, descrevem-se os procedimentos metodológicos, isto é, as etapas percorridas no desenvolvimento. A Seção 3.2.1 detalha a coleta de dados junto à fonte e caracteriza o processo de agendamento hoje em vigor no campus. A Seção 3.2.2 apresenta o levantamento e a análise dos requisitos funcionais e não funcionais. A Seção 3.3 trata da modelagem do sistema com UML. A Seção 3.3.1 apresenta o planejamento da implementação, com as tecnologias e a arquitetura definidas. A implementação propriamente dita do sistema, bem como os diagramas de classes e de sequência, serão conduzidos no TCC 2, conforme o cronograma da Seção 4.

3.1 TIPO DE PESQUISA

Quanto à natureza, esta pesquisa é aplicada. Sua finalidade é produzir um artefato de software para um problema concreto identificado num contexto institucional real, não ampliar o conhecimento teórico de forma genérica. Gil (2008) distingue a pesquisa aplicada

da pesquisa básica precisamente por esse critério: a primeira é motivada pela necessidade de resolver problemas práticos e seus resultados têm aplicação imediata.

Quanto aos objetivos, a pesquisa é descritiva. O ponto de partida é a caracterização do processo atual de agendamento de espaços físicos no IFNMG – Campus Salinas: como ele funciona, onde falha e o que os usuários envolvidos precisam que mude. Gil (2008) define a pesquisa descritiva como aquela que busca descrever as características de determinado fenômeno e estabelecer relações entre suas variáveis, sem que o pesquisador interfira nos fatos observados.

Quanto ao procedimento técnico, a pesquisa adota o estudo de caso em combinação com pesquisa bibliográfica. O estudo de caso é adequado porque a investigação recai sobre um ambiente específico, o Campus Salinas, com histórico, atores e restrições próprios que precisam ser compreendidos antes de qualquer proposta de solução. Yin (2015) define o estudo de caso como uma investigação empírica que examina um fenômeno contemporâneo em profundidade e em seu contexto real, sendo especialmente indicado quando os limites entre o fenômeno e o contexto não são claramente separáveis. A pesquisa bibliográfica, por sua vez, dá sustentação às decisões de projeto, de modelagem e de escolha tecnológica, com base no que a literatura já consolidou sobre engenharia de software, controle de acesso e desenvolvimento web.

3.2 PROCEDIMENTOS METODOLÓGICOS

Esta pesquisa, no âmbito do TCC, está organizada em quatro etapas encadeadas, cada uma condicionada pelos resultados da etapa anterior: o diagnóstico do problema, o levantamento e a análise de requisitos, a modelagem do sistema e a implementação. Atualmente, já foram realizadas as três primeiras etapas: o diagnóstico do processo atual de agendamento, a especificação dos requisitos funcionais e não funcionais e a modelagem do escopo funcional por meio do diagrama de casos de uso. A quarta e última etapa é a implementação, que permanece para o TCC 2 e compreende a codificação das funcionalidades especificadas, a elaboração dos diagramas de classes e de sequência e a realização dos testes, conforme o cronograma. Logo abaixo, é descrito o detalhamento de cada uma das etapas citadas.

3.2.1 DIAGNÓSTICO DO PROBLEMA

A primeira etapa consistiu no diagnóstico do problema. Por meio de conversas semiestruturadas, interações técnicas informais e conferências contínuas com a profissional responsável pela gestão dos espaços físicos no campus, foram mapeados o processo atual de agendamento, suas limitações operacionais e as necessidades concretas dos perfis envolvidos: Administrador e Docente, além do acesso público de consulta (Visualizador), disponível sem

autenticação. Essa etapa produziu o insumo necessário para a especificação dos requisitos; o processo atual de agendamento, tal como levantado, é caracterizado ao final desta seção.

Os dados desta pesquisa são primários, coletados diretamente junto à fonte por meio de um levantamento iterativo de requisitos, conduzido na forma de conversas semiestruturadas, interações técnicas informais, trocas de mensagens e discussões recorrentes sobre o processo de agendamento, além de conferências contínuas da proposta ao longo do desenvolvimento. Gil (2008) aponta a entrevista como uma das técnicas de coleta de dados mais utilizadas nas ciências sociais por permitir que o entrevistador obtenha informações sobre fatos, opiniões, percepções e necessidades de forma direta, com abertura para explorar aspectos não previstos no roteiro inicial. Neste trabalho, essa característica foi estendida para um formato iterativo, em que o diálogo com a fonte se prolongou para além de sessões pontuais.

As interações foram conduzidas com a Diretora de Ensino Técnico do IFNMG – Campus Salinas, coorientadora deste trabalho, que responde pela gestão dos agendamentos de espaços físicos no campus. Como marcos desse processo, foram realizadas três sessões mais estruturadas ao longo da pesquisa: em julho de 2025, novembro de 2025 e abril de 2026. Cada sessão cumpriu objetivos distintos. A primeira, em julho de 2025, teve como foco o mapeamento do processo atual de agendamento: como as reservas são feitas, quem as gerencia, quais conflitos ocorrem com mais frequência e que informações os usuários precisam acessar. A segunda, em novembro de 2025, foi dedicada à conferência dos requisitos preliminares levantados: as informações coletadas inicialmente foram apresentadas de forma estruturada para que a coorientadora confirmasse, corrigisse ou complementasse o entendimento do pesquisador. A terceira, em abril de 2026, serviu para confirmar os requisitos finais e verificar aspectos que surgiram durante a modelagem e ainda careciam de esclarecimento. Entre essas sessões, manteve-se interação técnica informal contínua – por trocas de mensagens e por reuniões breves, presenciais e remotas – para esclarecer dúvidas pontuais sobre o funcionamento do processo e confirmar decisões de modelagem à medida que surgiam.

A opção por esse formato semiestruturado e iterativo se justifica metodologicamente porque o roteiro funcionou como guia, sem impedir que temas relevantes surgidos no decorrer das conversas fossem explorados em profundidade, e porque permitiu que a compreensão do processo se refinasse de forma incremental, acompanhando o avanço da modelagem. Esse formato é mais adequado do que questionários fechados ou entrevistas rigidamente estruturadas quando o objetivo é compreender um processo operacional concreto, com suas particularidades e exceções, cujas nuances só emergem na medida em que a proposta de solução se concretiza.

O diagnóstico assim conduzido caracterizou o processo de agendamento hoje em vigor no campus, descrito a seguir.

Em termos operacionais, o processo percorre as etapas descritas a seguir. Antes do início de cada semestre, o setor responsável pela elaboração da grade produz as planilhas de horários, que registram, para cada turma, as aulas regulares e os espaços em que elas ocorrem. A partir dessas planilhas, a servidora responsável pelo agendamento cadastra manualmente, no sistema em uso, cada horário já ocupado pelas aulas, de modo que esses períodos deixem de aparecer como livres. Somente depois desse cadastro o sistema passa a refletir a ocupação real e pode receber pedidos de reserva. O docente que precisa de um laboratório ou de uma sala especial procura a servidora responsável, que consulta a disponibilidade, verifica se há conflito com outra atividade já registrada e efetiva a reserva. A confirmação retorna ao solicitante pelo mesmo canal informal pelo qual o pedido foi feito, e o registro do que está agendado permanece restrito a quem opera o sistema.

Esse arranjo apresenta as limitações que motivam a proposta deste trabalho. O cadastro manual descrito precisa ser refeito a cada troca de semestre, uma vez que as atividades dos espaços variam significativamente ao longo do período letivo, o que transforma o início de cada semestre em um período de retrabalho concentrado sobre uma única pessoa.

Além do retrabalho periódico, os docentes não têm acesso direto às informações de disponibilidade nem autonomia para solicitar reservas sem passar por um servidor responsável. Não há, tampouco, registro consolidado sobre a frequência e os padrões de uso dos laboratórios, limitando a capacidade de planejamento da equipe administrativa. Esse conjunto de restrições é o problema que o sistema proposto busca resolver.

Do ponto de vista da gestão, a ausência de dados históricos sobre o uso dos laboratórios impede a tomada de decisões com base em evidências. Sem saber quais espaços são mais demandados, em quais turnos, e quais ficam ociosos com frequência, a administração do campus não tem como planejar a alocação de recursos nem identificar gargalos na oferta de espaços. A questão extrapola, portanto, a simples reserva de ambientes: falta à instituição um mapa consolidado da utilização dos espaços do campus, capaz de tornar visível a ocupação real ao longo do semestre e de subsidiar decisões de planejamento que hoje são tomadas sem respaldo em dados. O processo atual concentra toda a responsabilidade de controle em um único ponto, o servidor que gerencia os agendamentos, situação que representa um risco operacional quando esse servidor está ausente ou sobrecarregado.

A Diretora de Ensino Técnico identificou ainda duas necessidades que o processo atual não atende. A primeira é a comunicação automática: no momento em que uma reserva é solicitada, o responsável pelo espaço precisa ser notificado de imediato, sem depender de contato manual. A segunda é a transparência sobre o que está agendado: docentes e demais membros da comunidade acadêmica precisam poder consultar os agendamentos de forma autônoma, sem precisar acionar a equipe administrativa. Essas duas necessidades orientam diretamente a especificação dos requisitos do sistema, em especial o módulo de

notificações por e-mail e o perfil de Visualizador, que permite acesso público de consulta sem permissões de edição.

Do diagnóstico decorre a decisão de projeto que orienta a solução proposta. A integração com o sistema de horários adotado pelo campus é o ponto de partida dessa solução. Em vez de exigir o cadastro manual sala por sala, o sistema importa as planilhas institucionais de horários já produzidas pelo setor responsável pela elaboração da grade, lê os dados e identifica automaticamente os horários ocupados pelas aulas regulares, marcando-os como indisponíveis para reserva. Esse mecanismo é o que torna viável, do ponto de vista operacional, substituir o processo atual: o gargalo deixa de ser o cadastro repetido a cada semestre e passa a ser uma operação de importação realizada pelo administrador.

Como o ensalamento pode sofrer alterações ao longo do semestre, entre remanejamentos de turmas, substituições de docentes, atividades extraordinárias e ajustes pontuais na grade, o sistema precisa lidar com a manutenção contínua da ocupação institucional. Para isso, o administrador poderá reimportar a planilha atualizada ou editar pontualmente os horários ocupados a partir da semana em que a alteração passa a valer, mantendo a base de disponibilidade consistente até o fim do período letivo. Cabe deixar explícito, contudo, que o sistema não consulta sistemas externos em tempo real: a atualização da ocupação institucional ocorre mediante ação administrativa, por importação ou edição, conduzida pelo responsável, preservando a autonomia da aplicação em relação à infraestrutura de horários do campus e evitando dependência de integrações automáticas com sistemas de terceiros.

3.2.2 LEVANTAMENTO E ANÁLISE DE REQUISITOS

A segunda etapa foi o levantamento e a análise de requisitos. Pressman e Maxim (2016) descrevem a engenharia de requisitos como um processo contínuo de identificação, análise, especificação e validação das necessidades dos envolvidos, e ressaltam que a qualidade dessa etapa determina a qualidade do que será desenvolvido depois. A especificação de requisitos foi elaborada a partir das informações coletadas nas entrevistas e do referencial bibliográfico consultado, e organizada em duas categorias: requisitos funcionais e requisitos não funcionais.

Os requisitos funcionais descrevem o que o sistema deve fazer, ou seja, as funções e os serviços que ele precisa oferecer a cada perfil de usuário (SOMMERVILLE, 2018). Para a especificação dessas funções, foram considerados os dois perfis autenticados identificados no diagnóstico, além do acesso público de consulta. O perfil Administrador concentra as funções de gestão: cadastro e manutenção de espaços físicos, gerenciamento de usuários, importação das planilhas de horários institucionais e atualização periódica dos horários ocupados, aprovação ou rejeição de solicitações de reserva e geração de relatórios sobre a utilização dos espaços. O perfil Docente pode consultar a disponibilidade dos espaços

e submeter solicitações de reserva, recebendo notificação automática sobre o resultado. A consulta de disponibilidade é acessível publicamente, sem necessidade de autenticação, exibindo apenas os horários livres e ocupados de cada espaço. Os requisitos funcionais foram agrupados por módulo: gestão de usuários e controle de acesso; gerenciamento de espaços físicos com importação de planilhas; agendamento e reservas; notificações automáticas por e-mail; e geração de relatórios de uso.

Os requisitos não funcionais estabelecem as restrições de qualidade que o sistema precisa satisfazer, independentemente das funcionalidades que oferece (SOMMERVILLE, 2018). Para defini-los, foram adotados como referência os atributos de qualidade descritos por Sommerville (2018): segurança, que implica controlar o acesso de acordo com os papéis definidos e proteger os dados dos usuários contra acessos indevidos; desempenho, que envolve tempo de resposta adequado nas consultas de disponibilidade, mesmo em picos de uso; manutenibilidade, que orienta decisões de projeto para facilitar a evolução do sistema após a implantação; e aceitabilidade, traduzida em requisitos de usabilidade compatíveis com o perfil tecnológico dos docentes do campus. Pressman e Maxim (2016) reforçam que os requisitos não funcionais condicionam a arquitetura do sistema e devem ser considerados desde o início do projeto, não como ajustes posteriores.

Para sistematizar a especificação descrita nos parágrafos anteriores, os requisitos funcionais e não funcionais foram consolidados em forma tabular. A Tabela 2 apresenta os requisitos funcionais, identificados pelo prefixo RF, e a Tabela 3 apresenta os requisitos não funcionais, identificados pelo prefixo RNF.

Tabela 2 – Requisitos funcionais do sistema

Código	**Descrição**
RF01	Importar planilhas de horários institucionais para identificar automaticamente os espaços e horários ocupados
RF02	Atualizar periodicamente os horários ocupados ao longo do semestre, por reimportação ou edição pontual realizada pelo administrador
RF03	Cadastrar e manter o registro dos espaços físicos do campus
RF04	Gerenciar usuários e seus perfis de acesso (Administrador, Docente)
RF05	Consultar a disponibilidade dos espaços físicos
RF06	Submeter solicitações de reserva de espaços físicos
RF07	Aprovar ou rejeitar solicitações de reserva
RF08	Cancelar reservas já registradas
RF09	Notificar automaticamente o administrador, por e-mail, quando uma nova solicitação de reserva for submetida
RF10	Notificar automaticamente o docente, por e-mail, sobre o resultado da análise da sua solicitação
RF11	Disponibilizar acesso público de consulta à disponibilidade dos espaços, sem autenticação e sem expor dados pessoais de docentes ou detalhes de reservas individuais
RF12	Gerar relatórios sobre a utilização dos espaços físicos

Tabela 3 – Requisitos não funcionais do sistema

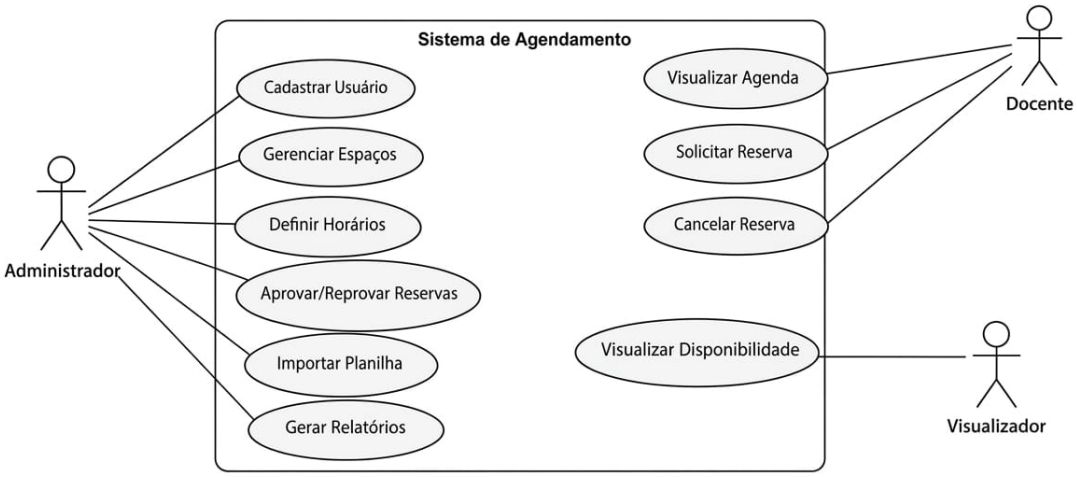
Código	Descrição
RNF01	O sistema deve controlar o acesso com base em papéis, conforme o modelo RBAC, restringindo cada funcionalidade ao perfil correspondente
RNF02	O sistema deve proteger os dados dos usuários contra acessos indevidos, incluindo proteções contra CSRF, XSS e injeção de SQL
RNF03	O sistema deve apresentar tempo de resposta adequado nas consultas de disponibilidade, mesmo em períodos de pico de uso
RNF04	O sistema deve ser acessível por meio de navegador web, sem exigir instalação de software nos dispositivos dos usuários
RNF05	O sistema deve apresentar usabilidade compatível com o perfil tecnológico dos docentes do campus e manter consistência visual entre suas telas, dispensando treinamento extensivo
RNF06	O sistema deve ser manutenível, de modo a permitir sua evolução após a implantação, com separação clara entre as camadas de dados, lógica e apresentação
RNF07	O sistema deve operar de forma autônoma em relação à infraestrutura de horários do campus, sem depender de consultas em tempo real a sistemas externos

3.3 MODELAGEM DO SISTEMA

A modelagem adotou a *Unified Modeling Language* (UML) para representar, de forma padronizada, a estrutura e o comportamento da solução antes da implementação. Conforme Bezerra (2007), a UML não é uma metodologia de desenvolvimento, mas uma notação que pode ser usada com diferentes processos; ela oferece um conjunto de diagramas que cobrem aspectos distintos do sistema. Nesta etapa do trabalho (TCC 1), foi elaborado o diagrama de casos de uso, que delimita o escopo funcional do sistema. Os diagramas de classes e de sequência serão construídos no TCC 2, na fase de implementação, a partir do escopo aqui definido.

O diagrama de casos de uso descreve as funcionalidades do sistema do ponto de vista dos atores que interagem com ele. Para os dois perfis autenticados (Administrador e Docente) foram associados os casos de uso correspondentes aos seus privilégios de acesso, com base no modelo RBAC descrito na Seção 2.3.4; o Visualizador representa o acesso público de consulta, sem autenticação. O Administrador concentra as operações de gestão: cadastrar usuários, gerenciar espaços físicos, definir horários, aprovar ou reprovar reservas, importar planilhas de horários institucionais e atualizar periodicamente os horários ocupados, e gerar relatórios de uso. O Docente pode visualizar a agenda, solicitar reservas e cancelar reservas já registradas. A consulta de disponibilidade é acessível por qualquer usuário sem autenticação, por meio de rota pública, sem permissão para solicitar ou alterar qualquer dado. O diagrama completo é apresentado na Figura 4.

Figura 4 – Diagrama de Casos de Uso do Sistema de Agendamento



Fonte: Elaborado pelo autor (2026).

Os diagramas de classes e de sequência serão elaborados no TCC 2. O primeiro terá como entidades centrais do domínio *EspaçoFísico*, *Horário*, *Solicitação*, *Reserva* e *Usuário*, cujos relacionamentos definirão as regras de integridade que orientam a estrutura do banco de dados; o segundo detalhará as interações entre os objetos nas operações de solicitação e de aprovação de reservas.

3.3.1 IMPLEMENTAÇÃO

A quarta e última etapa é a implementação. Nesta fase do trabalho apresenta-se o seu planejamento, com a definição das tecnologias e da arquitetura, enquanto a codificação propriamente dita será realizada no TCC 2. O sistema será desenvolvido como uma aplicação web, com arquitetura e tecnologias definidas a seguir.

A linguagem de programação adotada é o Python e o *framework* web escolhido é o Django. Seu funcionamento, que abrange os princípios *batteries included* e DRY, o roteamento de URLs, o ORM, as *migrations*, o sistema de *templates* e as proteções de segurança nativas, está detalhado na Seção 2.3.7. A opção pelo Django se justifica por reunir nativamente os componentes necessários ao sistema proposto, o que reduz a dependência de bibliotecas externas e o esforço de infraestrutura, permitindo que o desenvolvimento se concentre nas regras de negócio da aplicação.

O Django adota o padrão arquitetural MTV (*Model-Template-View*), cuja estrutura e comparação com o padrão MVC estão detalhadas na Seção 2.3.6. A adoção desse padrão no sistema proposto garante separação clara entre as camadas de dados, lógica e apresentação, facilitando a manutenção e a evolução do código ao longo do TCC 2.

Para a camada de dados, será utilizado o PostgreSQL como sistema gerenciador de banco de dados relacional. A escolha se justifica frente a alternativas como o MySQL

e o SQLite: embora o SQLite seja prático para ambientes de desenvolvimento, não é recomendado para aplicações com múltiplos acessos simultâneos em produção; o MySQL, por sua vez, é amplamente difundido, mas o PostgreSQL oferece maior conformidade com o padrão SQL, melhor tratamento de integridade referencial e suporte mais completo a transações complexas, recursos relevantes para a consistência das reservas. Soma-se a isso o suporte nativo e robusto do Django ao PostgreSQL, incluindo recursos como busca textual avançada e transações com alto nível de integridade (Django Software Foundation, 2024). O acesso ao banco será feito exclusivamente por meio do ORM, mantendo consistência entre o código Python e o esquema relacional ao longo de todo o ciclo de desenvolvimento.

A camada de apresentação será construída com os *templates* do Django combinados ao Tailwind CSS. Essa escolha agiliza o desenvolvimento da interface, mantém a consistência visual entre as telas e reduz o custo de manutenção, permitindo ajustes visuais sem alteração nas *views* ou nos *models*.

Para a autenticação dos usuários, será utilizada a biblioteca *django-allauth*, escolhida por fornecer prontos os fluxos de registro, login, recuperação de senha e gerenciamento de sessões (PENNERSR, 2024). O controle de acesso baseado em papéis, fundamentado no modelo RBAC descrito na Seção 2.3.4, será implementado sobre esse mecanismo, associando cada usuário a um dos dois perfis autenticados e condicionando o acesso a cada funcionalidade ao papel atribuído (Django Software Foundation, 2024).

As notificações automáticas serão enviadas por meio do módulo de e-mail nativo do Django, configurado com o protocolo SMTP e o serviço Gmail como servidor de envio (Django Software Foundation, 2024). Ao receber uma solicitação de reserva, o sistema notificará o administrador; após a decisão de aprovação ou rejeição, o docente receberá a confirmação correspondente, sem necessidade de contato manual entre as partes.

O planejamento da implementação contempla ainda a definição da infraestrutura de hospedagem do sistema. Essa questão foi levantada junto à Diretora de Ensino Técnico, que informou estar em andamento uma articulação da direção do *campus* junto à FADETEC e à Sifsoft, empresa júnior do curso de Bacharelado em Sistemas de Informação do *campus* Salinas. Como essa articulação ainda não está concretizada, prevê-se, como alternativa, a hospedagem em plataformas gratuitas ou de baixo custo voltadas a aplicações web, a exemplo de Render, Railway ou PythonAnywhere, que oferecem suporte a aplicações Django e permitem disponibilizar o sistema sob um domínio público sem custo de infraestrutura para a instituição.

Cabe esclarecer o escopo deste ponto: o presente trabalho compreende o desenvolvimento e a disponibilização de uma versão funcional do sistema em ambiente acessível por navegador. A implantação definitiva na infraestrutura institucional ficará a cargo da direção do campus e dos setores envolvidos.

Cabe delimitar igualmente o escopo da verificação. A conformidade do sistema com os requisitos especificados será verificada por testes unitários e de integração, conforme previsto no cronograma da Seção 4, cobrindo as regras de negócio centrais: a detecção de conflitos de horário, a checagem de permissões por perfil e o processamento das planilhas importadas. Não estão previstas, neste trabalho, a validação com usuários, a avaliação de usabilidade em ambiente real nem a homologação institucional do sistema, atividades que não cabem no cronograma do TCC. A avaliação dos ganhos obtidos com a adoção do sistema pelo campus fica registrada como trabalho futuro.

4 CRONOGRAMA

O cronograma a seguir contempla apenas a quarta e última etapa da pesquisa de implementação do sistema ao longo do segundo semestre de 2026. O desenvolvimento concentra-se entre agosto e novembro, de modo que a escrita e a entrega do TCC ocorram com a defesa, prevista para dezembro. As atividades foram distribuídas de forma a respeitar a dependência entre as camadas do sistema: a configuração do ambiente e a modelagem do banco de dados precedem o desenvolvimento das funcionalidades, que por sua vez antecedem os testes e os ajustes finais. A Tabela 4 apresenta essa distribuição.

Tabela 4 – Cronograma de implementação – 2º semestre de 2026

Etapa	**Meses**				
	Ago	Set	Out	Nov	Dez
Configuração do ambiente e estrutura do projeto	X				
Implementação dos *Models* e banco de dados	X	X			
Desenvolvimento das *Views* e roteamento de URLs		X			
Implementação dos *Templates* e interface		X	X		
Autenticação e controle de acesso (RBAC)		X			
Módulo de importação de planilhas de horários			X		
Módulo de notificações automáticas por e-mail			X		
Geração de relatórios de uso				X	
Testes unitários e de integração			X	X	
Implantação em ambiente de hospedagem				X	
Ajustes finais e escrita do TCC 2				X	
Defesa					X

REFERÊNCIAS

BEZERRA, E. *Princípios de Análise e Projeto de Sistemas com UML*. 2. ed. Rio de Janeiro: Elsevier, 2007. ISBN 9788535216967. Citado 2 vezes nas páginas 8 e 22.

CRUZ, I. N.; BARROSO, D. V. Gestão de laboratórios educacionais: análise da utilização de espaços e equipamentos do IFNMG Campus Almenara/MG. *RECITAL – Revista de Educação, Ciência e Tecnologia de Almenara*, v. 5, n. 1, 2023. Acesso em: abr. 2025. Disponível em: <https://recital.almenara.ifnmg.edu.br/index.php/recital/article/download/270/151>. Citado 3 vezes nas páginas 2, 4 e 6.

Django Software Foundation. *Django Documentation*. 2024. Django Project. Acesso em: abr. 2025. Disponível em: <https://docs.djangoproject.com/>. Citado 5 vezes nas páginas 11, 12, 13, 14 e 24.

GIL, A. C. *Métodos e Técnicas de Pesquisa Social*. 6. ed. São Paulo: Atlas, 2008. ISBN 9788522451425. Citado 3 vezes nas páginas 16, 17 e 18.

HEUSER, C. A. *Projeto de Banco de Dados*. 6. ed. Porto Alegre: Bookman, 2009. ISBN 9788577803828. Citado na página 8.

Instituto Federal do Norte de Minas Gerais. *Plano de Desenvolvimento Institucional 2024–2028*. Montes Claros: [s.n.], 2024. IFNMG. Acesso em: abr. 2025. Disponível em: <https://www.ifnmg.edu.br/pdi>. Citado na página 4.

LARMAN, C. *Utilizando UML e Padrões: Uma Introdução à Análise e ao Projeto Orientados a Objetos e ao Desenvolvimento Iterativo*. 3. ed. Porto Alegre: Bookman, 2007. ISBN 9788560031528. Citado na página 8.

LAUDON, K. C.; LAUDON, J. P. *Sistemas de Informação Gerenciais*. 11. ed. São Paulo: Pearson, 2014. ISBN 9788543004792. Citado 2 vezes nas páginas 10 e 11.

Microsoft. *Microsoft Bookings*. 2025. Microsoft. Acesso em: 4 jun. 2026. Disponível em: <https://www.microsoft.com/microsoft-365/business/scheduling-and-booking-app>. Citado 2 vezes nas páginas 2 e 5.

MYERS, G. J.; SANDLER, C.; BADGETT, T. *The Art of Software Testing*. 3. ed. Hoboken: Wiley, 2011. ISBN 9781118031964. Citado na página 16.

NASCIMENTO, T. A. d.; RIOS, E. F. S.; BALUZ, R. A. R. S. Sistema Web para Controle dos Pedidos de Reservas de Recursos em Instituições de Ensino Superior. *Revista Eletrônica de Iniciação Científica em Computação*, v. 18, 2020. Acesso em: abr. 2025. Disponível em: <https://journals-sol.sbc.org.br/index.php/reic/article/view/1721>. Citado 4 vezes nas páginas 2, 4, 6 e 7.

NIELSEN, J. *Usability Engineering*. Boston: Academic Press, 1993. ISBN 9780125184069. Citado na página 14.

NIELSEN, J. *10 Usability Heuristics for User Interface Design*. 1994. Nielsen Norman Group. Acesso em: abr. 2025. Disponível em: <https://www.nngroup.com/articles/ten-usability-heuristics/>. Citado na página 15.

PENNERSR, R. *django-allauth Documentation*. 2024. Allauth Project. Acesso em: abr. 2025. Disponível em: <https://docs.allauth.org/>. Citado na página 24.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software: Uma Abordagem Profissional*. 8. ed. Porto Alegre: AMGH, 2016. ISBN 9788580555349. Citado 4 vezes nas páginas 15, 16, 20 e 21.

SANDHU, R. S. et al. Role-based access control models. *IEEE Computer*, v. 29, n. 2, p. 38–47, 1996. Citado 2 vezes nas páginas 9 e 10.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. *Sistema de Banco de Dados*. 6. ed. Rio de Janeiro: Elsevier, 2012. ISBN 9788535245356. Citado 2 vezes nas páginas 8 e 9.

SILVA, D. *Como funciona a arquitetura MTV – Django*. 2023. Medium. Acesso em: abr. 2025. Disponível em: <https://diandrasilva.medium.com/como-funciona-a-arquitetura-mtv-django-86af916f1f63>. Citado na página 13.

Skedda. *Skedda: Space Booking and Scheduling Software*. 2025. Skedda. Acesso em: 4 jun. 2026. Disponível em: <https://www.skedda.com>. Citado 2 vezes nas páginas 2 e 5.

SOMMERVILLE, I. *Engenharia de Software*. 10. ed. São Paulo: Pearson, 2018. ISBN 9788543024974. Citado 6 vezes nas páginas 7, 8, 15, 16, 20 e 21.

TANENBAUM, A. S.; WETHERALL, D. J. *Redes de Computadores*. 5. ed. São Paulo: Pearson, 2011. ISBN 9788576059240. Citado na página 10.

The PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2024. The PostgreSQL Global Development Group. Acesso em: 4 jun. 2026. Disponível em: <https://www.postgresql.org/docs/>. Citado na página 9.

TreinaWeb. *Entendendo o MTV do Django*. 2023. TreinaWeb Blog. Acesso em: abr. 2025. Disponível em: <https://www.treinaweb.com.br/blog/entendendo-o-mtv-do-django>. Citado 2 vezes nas páginas 11 e 12.

YIN, R. K. *Estudo de Caso: Planejamento e Métodos*. 5. ed. Porto Alegre: Bookman, 2015. ISBN 9788582602324. Citado na página 17.